

Lecture Notes on Tree & Graph

CSE207: Data Structures

Ahmed Abdal Shafi Rasel
Senior Lecturer, Department of CSE
East West University

1 Part 1: Binary Tree Methods

1.1 Question 1: Write createNode() Function

Given: The following Node structure for a binary tree:

```
struct Node {  
    int data;  
    struct Node* left;  
    struct Node* right;  
};
```

Task: Write a function `createNode(int value)` that creates and returns a new node with the given value. The left and right pointers should be initialized to NULL.

Answer:

```
struct Node* createNode(int value) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node)  
    );  
    newNode->data = value;  
    newNode->left = NULL;  
    newNode->right = NULL;  
    return newNode;  
}
```

1.2 Question 2: Complete the Pre-order Traversal Function

Given: A binary tree with the following structure:

```
struct Node {  
    int data;  
    struct Node* left;  
    struct Node* right;  
};
```

Task: Complete the `preorder()` function to traverse the tree in pre-order (Root → Left → Right). Print each node's data.

```
void preorder(struct Node* root) {  
    // Write your code here  
}
```

Answer:

```
void preorder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}
```

1.3 Question 3: Complete the In-order Traversal Function

Given: A binary tree structure:

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
```

Task: Complete the `inorder()` function to traverse the tree in in-order (Left $\rightarrow$ Root $\rightarrow$ Right).

```
void inorder(struct Node* root) {
    // Write your code here
}
```

Answer:

```
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
```

1.4 Question 4: Complete the Post-order Traversal Function

Given: A binary tree structure:

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
```

Task: Complete the `postorder()` function to traverse the tree in post-order (Left $\rightarrow$ Right $\rightarrow$ Root).

```
void postorder(struct Node* root) {
    // Write your code here
}
```

Answer:

```
void postorder(struct Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}
```

1.5 Question 5: Write Tree Height Function

Given: A binary tree structure and a max helper function:

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

int max(int a, int b) {
    return (a > b) ? a : b;
}
```

Task: Write a recursive function `height(struct Node* root)` that returns the height of the tree. Height is the number of edges on the longest path from root to a leaf. An empty tree has height -1.

Answer:

```
int height(struct Node* root) {
    if (root == NULL) {
        return -1;
    }

    int leftHeight = height(root->left);
    int rightHeight = height(root->right);

    return 1 + max(leftHeight, rightHeight);
}
```

1.6 Question 6: Complete the buildTree() Function

Given: The following starter code:

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node)
    );
}
```

```

    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

struct Node* buildTree() {
    int value;
    printf("Enter node value (-1 for no node): ");
    scanf("%d", &value);

    if (value == -1) {
        return NULL;
    }

    struct Node* node = createNode(value);
    char opt;

    // Complete the code here to ask for left and right children
}

```

Task: Complete the buildTree() function to ask the user if the node has left and right children, and recursively build the tree.

Answer:

```

struct Node* buildTree() {
    int value;
    printf("Enter node value (-1 for no node): ");
    scanf("%d", &value);

    if (value == -1) {
        return NULL;
    }

    struct Node* node = createNode(value);
    char opt;

    printf("Does node %d have a left child? (y/n): ", value);
    scanf(" %c", &opt);
    if (opt == 'y') {
        node->left = buildTree();
    }

    printf("Does node %d have a right child? (y/n): ", value);
    scanf(" %c", &opt);
    if (opt == 'y') {
        node->right = buildTree();
    }

    return node;
}

```

2 Part 2: Adjacency Matrix Methods

2.1 Question 7: Write addEdge() for Adjacency Matrix

Given: A 2D array representing an adjacency matrix for a directed graph:

```
int adjMatrix[100][100];
```

Task: Write a function `addEdge(int matrix[][100], int src, int dest)` that adds a directed edge from source to destination in the adjacency matrix.

Answer:

```
void addEdge(int matrix[][100], int src, int dest) {  
    matrix[src][dest] = 1;  
}
```

2.2 Question 8: Write removeEdge() for Adjacency Matrix

Given: A 2D array representing an adjacency matrix:

```
int adjMatrix[100][100];
```

Task: Write a function `removeEdge(int matrix[][100], int src, int dest)` that removes a directed edge from source to destination.

Answer:

```
void removeEdge(int matrix[][100], int src, int dest) {  
    matrix[src][dest] = 0;  
}
```

2.3 Question 9: Write displayMatrix() Function

Given: A 2D adjacency matrix:

```
int adjMatrix[100][100];  
int vertices = 6;
```

Task: Write a function `displayMatrix(int matrix[][100], int vertices)` that prints the adjacency matrix in a formatted way with row and column indices.

Answer:

```
void displayMatrix(int matrix[][100], int vertices) {  
    printf("Adjacency Matrix:\n");  
  
    // Print column headers  
    for (int i = 0; i < vertices; i++) {  
        printf("%2d ", i);  
    }  
    printf("\n");
```

```

// Print each row with row index
for (int i = 0; i < vertices; i++) {
    printf("%2d ", i);
    for (int j = 0; j < vertices; j++) {
        printf("%2d ", matrix[i][j]);
    }
    printf("\n");
}
}

```

2.4 Question 10: Write hasEdge() Function

Given: An adjacency matrix:

```
int adjMatrix[100][100];
```

Task: Write a function `hasEdge(int matrix[][100], int src, int dest)` that returns 1 if there is an edge from `src` to `dest`, otherwise returns 0.

Answer:

```

int hasEdge(int matrix[][100], int src, int dest) {
    return matrix[src][dest];
}

```

3 Part 3: Adjacency List Methods

3.1 Question 11: Write createNode() for Adjacency List

Given: The following Node structure for adjacency list:

```

struct Node {
    int dest;
    struct Node* next;
};

```

Task: Write a function `createNode(int dest)` that creates and returns a new list node with the given destination vertex.

Answer:

```

struct Node* createNode(int dest) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->dest = dest;
    newNode->next = NULL;
    return newNode;
}

```

3.2 Question 12: Complete createGraph() Function

Given: The following structures and partial code:

```
struct Node {
    int dest;
    struct Node* next;
};

struct Graph {
    int vertices;
    struct Node** adjList;
};

struct Graph* createGraph(int vertices) {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->vertices = vertices;

    // Complete the code here
}
```

Task: Complete the function to allocate memory for the adjacency list array and initialize all lists to NULL.

Answer:

```
struct Graph* createGraph(int vertices) {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->vertices = vertices;

    graph->adjList = (struct Node**)malloc(vertices * sizeof(struct Node*));

    for (int i = 0; i < vertices; i++) {
        graph->adjList[i] = NULL;
    }

    return graph;
}
```

3.3 Question 13: Write addEdge() for Directed Graph (Adjacency List)

Given: The following structures and helper function:

```
struct Node {
    int dest;
    struct Node* next;
};

struct Graph {
    int vertices;
    struct Node** adjList;
};

struct Node* createNode(int dest) {
```

```

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node)
    );
    newNode->dest = dest;
    newNode->next = NULL;
    return newNode;
}

```

Task: Write a function `addEdge(struct Graph* graph, int src, int dest)` that adds a directed edge from `src` to `dest` in the adjacency list.

Answer:

```

void addEdge(struct Graph* graph, int src, int dest) {
    struct Node* newNode = createNode(dest);
    newNode->next = graph->adjList[src];
    graph->adjList[src] = newNode;
}

```

3.4 Question 14: Write `addEdge()` for Undirected Graph (Adjacency List)

Given: The same structures as Question 13.

Task: Write a function `addEdge(struct Graph* graph, int src, int dest)` that adds an undirected edge between `src` and `dest` (add edges in both directions).

Answer:

```

void addEdge(struct Graph* graph, int src, int dest) {
    // Add edge from src to dest
    struct Node* newNode = createNode(dest);
    newNode->next = graph->adjList[src];
    graph->adjList[src] = newNode;

    // Add edge from dest to src
    newNode = createNode(src);
    newNode->next = graph->adjList[dest];
    graph->adjList[dest] = newNode;
}

```

3.5 Question 15: Complete `displayList()` Function

Given: The graph structure:

```

struct Node {
    int dest;
    struct Node* next;
};

struct Graph {
    int vertices;
    struct Node** adjList;
};

void displayList(struct Graph* graph) {

```

```

    printf("Adjacency List:\n");
    for (int i = 0; i < graph->vertices; i++) {
        printf("%d -> ", i);
        // Complete the code here
    }
}

```

Task: Complete the function to print all vertices adjacent to each vertex.

Answer:

```

void displayList(struct Graph* graph) {
    printf("Adjacency List:\n");
    for (int i = 0; i < graph->vertices; i++) {
        printf("%d -> ", i);
        struct Node* temp = graph->adjList[i];
        while (temp != NULL) {
            printf("%d ", temp->dest);
            temp = temp->next;
        }
        printf("NULL\n");
    }
}

```

3.6 Question 16: Write removeEdge() for Directed Graph (Adjacency List)

Given: The graph structure:

```

struct Node {
    int dest;
    struct Node* next;
};

struct Graph {
    int vertices;
    struct Node** adjList;
};

```

Task: Write a function removeEdge(struct Graph* graph, int src, int dest) that removes a directed edge from src to dest in the adjacency list.

Answer:

```

void removeEdge(struct Graph* graph, int src, int dest) {
    struct Node* temp = graph->adjList[src];
    struct Node* prev = NULL;

    // Find the node to delete
    while (temp != NULL && temp->dest != dest) {
        prev = temp;
        temp = temp->next;
    }

    // If edge not found
    if (temp == NULL) {
        return;
    }
}

```

```

    // Remove the node
    if (prev == NULL) {
        // Node to delete is the first node
        graph->adjList[src] = temp->next;
    } else {
        prev->next = temp->next;
    }

    free(temp);
}

```

4 Part 4: DFS Methods

4.1 Question 17: Complete DFS Function

Given: The following structures and partial code:

```

struct Node {
    int dest;
    struct Node* next;
};

struct Graph {
    int vertices;
    struct Node** adjList;
};

void dfs(struct Graph* graph, int vertex, int visited[]) {
    visited[vertex] = 1;
    printf("%d ", vertex);

    // Complete the code here to visit all unvisited neighbors
}

```

Task: Complete the DFS function to recursively visit all unvisited neighbors.

Answer:

```

void dfs(struct Graph* graph, int vertex, int visited[]) {
    visited[vertex] = 1;
    printf("%d ", vertex);

    struct Node* temp = graph->adjList[vertex];
    while (temp != NULL) {
        if (!visited[temp->dest]) {
            dfs(graph, temp->dest, visited);
        }
        temp = temp->next;
    }
}

```

4.2 Question 18: Complete printConnectedComponents() Function

Given: The graph structure and DFS helper:

```

struct Node {
    int dest;
    struct Node* next;
};

struct Graph {
    int vertices;
    struct Node** adjList;
};

void dfsComponent(struct Graph* graph, int vertex, int visited[]) {
    visited[vertex] = 1;
    printf("%d ", vertex);

    struct Node* temp = graph->adjList[vertex];
    while (temp != NULL) {
        if (!visited[temp->dest]) {
            dfsComponent(graph, temp->dest, visited);
        }
        temp = temp->next;
    }
}

void printConnectedComponents(struct Graph* graph) {
    int visited[100] = {0};
    int componentNum = 1;

    printf("Connected Components:\n");

    // Complete the code here
}

```

Task: Complete the function to find and print all connected components.

Answer:

```

void printConnectedComponents(struct Graph* graph) {
    int visited[100] = {0};
    int componentNum = 1;

    printf("Connected Components:\n");

    for (int v = 0; v < graph->vertices; v++) {
        if (!visited[v]) {
            printf("Component %d: ", componentNum++);
            dfsComponent(graph, v, visited);
            printf("\n");
        }
    }
}

```

5 Part 5: Utility Methods

5.1 Question 19: Complete isBalanced() Function

Given: The tree structure and height function:

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

int abs(int x) {
    return (x < 0) ? -x : x;
}

int height(struct Node* root) {
    if (root == NULL) return -1;
    int lh = height(root->left);
    int rh = height(root->right);
    return 1 + (lh > rh ? lh : rh);
}

int isBalanced(struct Node* root) {
    if (root == NULL) {
        return 1;
    }

    // Complete the code here
}
```

Task: Complete the function to check if the tree is balanced.

Answer:

```
int isBalanced(struct Node* root) {
    if (root == NULL) {
        return 1;
    }

    int leftHeight = height(root->left);
    int rightHeight = height(root->right);

    if (abs(leftHeight - rightHeight) > 1) {
        return 0;
    }

    return isBalanced(root->left) && isBalanced(root->right);
}
```

5.2 Question 20: Write countNodes() Function

Given: A binary tree structure:

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
```

Task: Write a recursive function `countNodes(struct Node* root)` that returns the total number of nodes in the tree.

Answer:

```
int countNodes(struct Node* root) {
    if (root == NULL) {
        return 0;
    }
    return 1 + countNodes(root->left) + countNodes(root->right);
}
```

5.3 Question 21: Write countLeaves() Function

Given: A binary tree structure:

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
```

Task: Write a function `countLeaves(struct Node* root)` that counts and returns the number of leaf nodes. A leaf node has no children.

Answer:

```
int countLeaves(struct Node* root) {
    if (root == NULL) {
        return 0;
    }

    // If it's a leaf node
    if (root->left == NULL && root->right == NULL) {
        return 1;
    }

    return countLeaves(root->left) + countLeaves(root->right);
}
```

5.4 Question 22: Complete matrixToList() Conversion

Given: Partial code for converting adjacency matrix to list:

```
struct Node {
    int dest;
    struct Node* next;
```

```

};

struct Node* createNode(int dest) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node)
    );
    newNode->dest = dest;
    newNode->next = NULL;
    return newNode;
}

struct Node** matrixToList(int matrix[][100], int vertices) {
    struct Node** adjList = (struct Node**)malloc(vertices * sizeof
    (struct Node*));

    for (int i = 0; i < vertices; i++) {
        adjList[i] = NULL;
    }

    // Complete the conversion logic here

    return adjList;
}

```

Task: Complete the function to convert the adjacency matrix to adjacency list.

Answer:

```

struct Node** matrixToList(int matrix[][100], int vertices) {
    struct Node** adjList = (struct Node**)malloc(vertices * sizeof
    (struct Node*));

    for (int i = 0; i < vertices; i++) {
        adjList[i] = NULL;
    }

    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            if (matrix[i][j] == 1) {
                struct Node* newNode = createNode(j);
                newNode->next = adjList[i];
                adjList[i] = newNode;
            }
        }
    }

    return adjList;
}

```

6 Exercises

The following exercises are left for students to practice. Try to solve them without looking at hints or solutions.

6.1 Binary Tree Exercises

1. **Write findMax() Function** - Find the maximum value in a binary tree
2. **Complete findDepth() Function** - Find depth of a specific node from root
3. **Write sumOfNodes() Function** - Calculate sum of all node values

6.2 Adjacency Matrix Exercises

1. **Write getDegree() Function** - Calculate degree (in-degree or out-degree) of a vertex
2. **Complete countEdges() Function** - Count total number of edges in the graph

6.3 Adjacency List Exercises

1. **Write hasEdge() Function** - Check if an edge exists between two vertices in adjacency list
2. **Complete getDegree() Function** - Count the degree of a vertex in adjacency list
3. **Complete printNeighbors() Function** - Print all neighbors of a given vertex